

## **MARTIN SILVA CABRERA**

Taller número 1 en el curso Lab. Ingeniería de Software II: Práctica de Polimorfismo

Profesora:  
Paola Andrea Bedoya Toro

**Universidad del Cauca**  
Facultad de Ingeniería Electrónica y Telecomunicaciones  
Departamento de Sistemas  
Ingeniería de Software II

Popayán, 2026

**MARTIN SILVA CABRERA**

**PRÁCTICA DE POLIMORFISMO**

Taller No. 1 presentado en el curso Lab. Ingeniería de Software II

Estudiante del Programa de Ingeniería de Sistemas

Profesor:  
Paola Andrea Bedoya Toro

**Universidad del Cauca**  
Popayán  
2026

# contenido

|                                                        |                               |
|--------------------------------------------------------|-------------------------------|
| 1. introducción.....                                   | 1                             |
| 1.1.- objetivo .....                                   | 1                             |
| 2. diseño del ejercicio .....                          | 1                             |
| 2.1.- diagrama de clases.....                          | ¡Error! Marcador no definido. |
| 3. implementación .....                                | 1                             |
| 3.1.- clase abstracta Figure .....                     | 2                             |
| 3.2.- clases concretas Triangle, Circle y Square ..... | 2                             |
| 3.3.- código fuente completo .....                     | 2                             |
| 4. repositorio de código.....                          | 4                             |
| 5. pruebas unitarias .....                             | 4                             |
| 5.1.- clase de prueba TriangleTest .....               | 4                             |
| 5.2.- clase de prueba CircleTest .....                 | 4                             |
| 5.3.- clase de prueba SquareTest .....                 | 5                             |
| 6. resultados de ejecución.....                        | 5                             |
| 7. conclusiones .....                                  | 6                             |

## **lista de figuras**

Figura 6.1 Evidencia de ejecución de las pruebas unitarias.

Figura 7.1 Evidencia de ejecución del programa (salida por consola).

## lista de tablas

Tabla 5.1 Pruebas unitarias propuestas para las clases Triangle, Circle y Square.

## **lista de siglas**

POO: Programación Orientada a Objetos.

UML: Unified Modeling Language (Lenguaje Unificado de Modelado).

JVM: Java Virtual Machine (Máquina Virtual de Java).

# práctica de polimorfismo — taller no. 1

## 1. introducción

Este documento presenta el desarrollo del Taller No. 1 del curso Lab. Ingeniería de Software II, cuyo propósito es ejemplificar el uso del principio de polimorfismo en el paradigma de la Programación Orientada a Objetos (POO), mediante la implementación en Java de una jerarquía de figuras geométricas. El polimorfismo permite que una misma referencia de tipo abstracto (Figure) invoque, en tiempo de ejecución, la implementación específica de cada subclase (Triangle, Circle, Square), evidenciando así el enlace dinámico (dynamic binding) característico de este principio.

### 1.1.- objetivo

Implementar y comprobar el comportamiento polimórfico de una jerarquía de clases geométricas en Java, a partir de una clase abstracta que define un contrato común (cálculo de área y perímetro), verificando su correcto funcionamiento a través de pruebas unitarias.

## 2. diseño del ejercicio

El ejercicio se basa en una clase abstracta Figure que declara dos métodos abstractos: calculateArea() y getPerimeter(). Tres clases concretas (Triangle, Circle y Square) extienden Figure y ofrecen su propia implementación de dichos métodos, de acuerdo con la fórmula geométrica que les corresponde. La clase Main construye una lista de tipo List<Figure> con instancias de las tres subclases y recorre dicha lista invocando calculateArea() y getPerimeter() sobre la referencia Figure; en cada iteración se ejecuta la versión sobrescrita correspondiente al tipo real del objeto, lo cual constituye el comportamiento polimórfico central del ejercicio.

## 3. implementación

A continuación se presenta el código fuente completo del ejercicio, escrito en Java. Se compone de la clase abstracta Figure, las clases concretas Triangle, Circle y Square, y la clase Main que ejecuta la demostración del polimorfismo.

### 3.1.- clase abstracta Figure

Define el contrato común de toda figura: los atributos genéricos x, y y los métodos abstractos calculateArea() y getPerimeter(), que obligan a cada subclase a proveer su propia implementación.

### 3.2.- clases concretas Triangle, Circle y Square

Cada clase concreta hereda de Figure y sobrescribe (@Override) los métodos abstractos: Triangle calcula el área como  $(base \times altura)/2$  y el perímetro sumando los tres lados (la hipotenusa se obtiene mediante el teorema de Pitágoras); Circle calcula el área como  $\pi \cdot r^2$  y el perímetro como  $2 \cdot \pi \cdot r$ ; Square calcula el área como  $lado^2$  y el perímetro como  $4 \cdot lado$ .

### 3.3.- código fuente completo

```
import java.util.ArrayList;
import java.util.List;
//clase abstracta Figura que define los métodos para calcular el área y el perímetro de
una figura geométrica.
abstract class Figure {
    double x;
    double y;

    abstract double calculateArea();
    abstract double getPerimeter();
}
//clase Triángulo que extiende la clase Figura y proporciona implementaciones concretas
de los métodos para calcular el área y el perímetro de un triángulo.
class Triangle extends Figure {

    Triangle(double x, double y) {
        this.x = x; // base
        this.y = y; // altura
    }

    @Override
    double calculateArea() {
        return (x * y) / 2.0;
    }

    @Override
    double getPerimeter() {
        double hipotenusa = Math.sqrt(Math.pow(x, 2) + Math.pow(y, 2));
        return x + y + hipotenusa;
    }
}
//clase Círculo que extiende la clase Figura y proporciona implementaciones concretas de
los métodos para calcular el área y el perímetro de un círculo.
class Circle extends Figure {
```

```

    Circle(double x) {
        this.x = x; // radio
    }

    @Override
    double calculateArea() {
        return Math.PI * x * x;
    }

    @Override
    double getPerimeter() {
        return 2 * Math.PI * x;
    }
}
//clase Cuadrado que extiende la clase Figura y proporciona implementaciones concretas de
los métodos para calcular el área y el perímetro de un cuadrado.
class Square extends Figure {

    Square(double side) {
        this.x = side; // lado
    }

    @Override
    double calculateArea() {
        return x * x;
    }

    @Override
    double getPerimeter() {
        return 4 * x;
    }
}

public class Main {
    public static void main(String[] args) {
        // Crear instancias de las figuras geométricas
        Figure fig1 = new Circle(1.0);
        Figure fig2 = new Square(2.3);
        Figure fig3 = new Triangle(4.2, 4.5);
        // Agregar las figuras a una lista y calcular el área y el perímetro de cada una
        List<Figure> figures = new ArrayList<>();
        figures.add(fig1);
        figures.add(fig2);
        figures.add(fig3);

        for (Figure fig : figures) {
            System.out.println("Area: " + fig.calculateArea());
            System.out.println("Perimeter: " + fig.getPerimeter());
            System.out.println("-----");
        }
    }
}

```

## 4. repositorio de código

El código fuente completo del ejercicio, junto con las pruebas unitarias, se encuentra publicado en el siguiente repositorio de GitHub:

(<https://github.com/martin1061/LaboratorioIngenieriaSoftware2>)

## 5. pruebas unitarias

Para verificar el correcto funcionamiento de cada implementación y del comportamiento polimórfico general, se proponen las siguientes pruebas unitarias (JUnit). En la Tabla 5.1 se listan las pruebas sugeridas junto con la condición que cada una valida; debajo de la tabla se deja el espacio para incluir la evidencia (capturas de pantalla) de su ejecución.

### 5.1.- clase de prueba TriangleTest

| Prueba unitaria                                  | Qué valida                                                                                                                                                                    |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| testCalculateArea_returnsHalfBaseTimesHeight     | Verifica que calculateArea() de Triangle retorne exactamente $(base \times altura) / 2$ para valores conocidos (por ejemplo, base = 4, altura = 6 $\rightarrow$ área = 12.0). |
| testGetPerimeter_returnsSumOfSidesWithHypotenuse | Verifica que getPerimeter() calcule correctamente la hipotenusa con el teorema de Pitágoras ( $\sqrt{base^2 + altura^2}$ ) y la sume a los dos catetos.                       |

### Datos Ingresados:

```
Figure fig3 = new Triangle(x: 4.2, y: 4.5);
```

### Salida:

```
Area: 9.450000000000001
Perimeter: 14.855485358604955
```

## 5.2.- clase de prueba CircleTest

| Prueba unitaria                     | Qué valida                                                                                                                       |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| testCalculateArea_returnsPiRSquared | Verifica que calculateArea() retorne $\pi \cdot r^2$ dentro de una tolerancia (delta) razonable para comparación de tipo double. |
| testGetPerimeter_returnsTwoPiR      | Verifica que getPerimeter() retorne $2 \cdot \pi \cdot r$ para el radio dado.                                                    |

### Datos ingresados:

```
Figure fig1 = new Circle(x: 1.0);
```

### Salida:

```
Area: 3.141592653589793  
Perimeter: 6.283185307179586
```

## 5.3.- clase de prueba SquareTest

| Prueba unitaria                       | Qué valida                                                                                 |
|---------------------------------------|--------------------------------------------------------------------------------------------|
| testCalculateArea_returnsSideSquared  | Verifica que calculateArea() retorne lado <sup>2</sup> para el valor de lado suministrado. |
| testGetPerimeter_returnsFourTimesSide | Verifica que getPerimeter() retorne $4 \times$ lado.                                       |

### Datos ingresados:

```
Figure fig2 = new Square(side: 2.3);
```

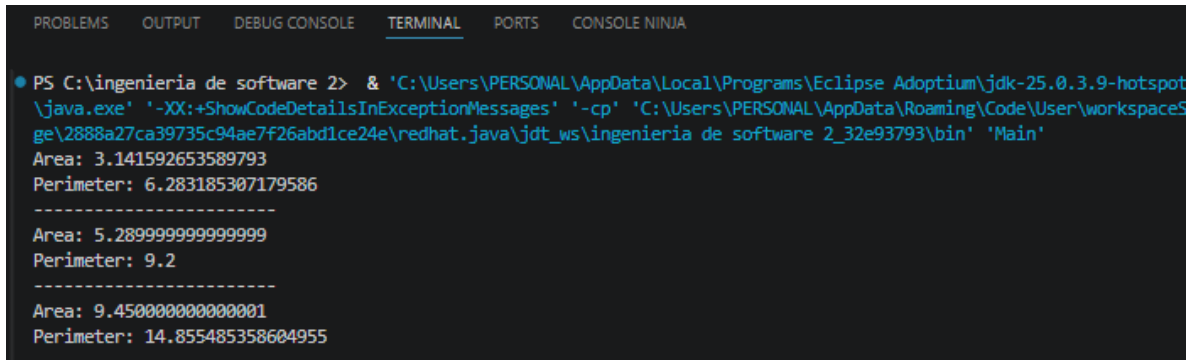
### Salida:

```
Area: 5.289999999999999  
Perimeter: 9.2
```

Figura 6.1 Evidencia de ejecución de las pruebas unitarias.

## 6. resultados de ejecución

A continuación se debe insertar la evidencia de la salida por consola del método main, donde se observa el área y el perímetro calculados para cada figura de la lista (Circle, Square y Triangle), confirmando el comportamiento polimórfico descrito en la Sección 2.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CONSOLE NINJA
PS C:\ingenieria de software 2> & 'C:\Users\PERSONAL\AppData\Local\Programs\Eclipse Adoptium\jdk-25.0.3-hotspot\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\PERSONAL\AppData\Roaming\Code\User\workspaceS
ge\2888a27ca39735c94ae7f26abd1ce24e\redhat.java\jdt_ws\ingenieria de software 2_32e93793\bin' 'Main'
Area: 3.141592653589793
Perimeter: 6.283185307179586
-----
Area: 5.289999999999999
Perimeter: 9.2
-----
Area: 9.450000000000001
Perimeter: 14.855485358604955
-----
```

Figura 7.1 Evidencia de ejecución del programa (salida por consola).

## 7. conclusiones

- El uso de una clase abstracta con métodos abstractos permite definir un contrato común que obliga a cada subclase a implementar su propio comportamiento, favoreciendo la extensibilidad del diseño (agregar una nueva figura no requiere modificar el código que ya usa Figure).
- El polimorfismo se evidencia al recorrer una `List<Figure>` e invocar `calculateArea()` y `getPerimeter()`: en tiempo de ejecución la JVM resuelve el método correspondiente al tipo real de cada objeto (enlace dinámico), no al tipo declarado de la referencia.
- Las pruebas unitarias propuestas permiten validar de forma aislada la fórmula matemática de cada figura y, mediante la prueba de la lista polimórfica, comprobar que el mecanismo de sobrescritura (`@Override`) funciona correctamente en conjunto.

## **bibliografía**

- [1] M. Petersen, K. Feldt, R. Mujtaba, and S. Mattsson, “Systematic Mapping Studies in Software Engineering,” Proc. 12th Int. Conf. Eval. Assess. Softw. Eng., pp. 68–77, 2008, doi: 10.1142/S0218194007003112.
- [2] Oracle, “Polymorphism (The Java Tutorials > Learning the Java Language > Interfaces and Inheritance),” Oracle Corporation. [En línea]. Disponible en documentación oficial de Java.